Objet	**Mode opératoire : réplication MySQL vers AWS Aurora**
Client	SPIE – PRA CMDB pour la PFS
Date	03/02/2025
Rédacteur	Jean-Christophe GASTAUD

DIFFUSION			
Nom	**Titre / Rôle**	**Société**	**diffusion**
Damien ROSEAU	Référent Technique	SPIE	√
Julien BOUKHELIFA	Team Leader BO	SPIE	√
Thomas SCHNEL	Responsable du BO	MSA	√

REDACTION			
Nom	**Tous les participants et :**	**Société**	**version**
Jean-christophe GASTAUD	Ingénieur infrastructure	SPIE ICS	1.0

MENU

AVANT-PROPOS 3
ANALYSE DE L'EXISTANT 4
PRE-REQUIS DBA 6
NOUVELLE INFRASTRUCTURE VPC 6
NOUVELLE INFRASTRUCTURE EC2 7
CONFIGURATION DE L'INFRASTRUCTURE RDS 11
Prérequis 11
MIGRATION – REPLICATION AVEC DMS 15
ARCHITECTURE PROPOSEE 15
INSTALLATION D'AURORA MYSQL COMME DB CIBLE 16
Construction de la stack à partir du fichier yaml 16
INSTALLATION DE L'INSTANCE AWS DMS POUR LA REPLICATION 18
Construction de la stack à partir du fichier yaml 18
CREATION DE TACHES AWS DMS 19
VALIDATION DU REPLICA AWS DMS 19
CUT OVER 19
ANNEXES 20
ARCHITECTURE DU VPC PRA-RMS (AWS) 20
REPLICATION CIRCULAIRE AVEC MYSQL 22
TEMPLATE AWS CLOUDFORMATION POUR LE DEPLOIEMENT D'AWS AURORA 23
TEMPLATE AWS CLOUDFORMATION POUR LE DEPLOIEMENT DE L'INSTANCE DMS 24
FONCTIONNALITES POUR UNE MIGRATION CDC 25
BUILD IAM ROLE(S) POUR DMS 26
AWS CLOUDFORMATION PILE DE CONSTRUCTION DB MYSQL POUR RDS 29

Avant-propos

Le CSM de DCSN a mis en place une solution PRA orientée cloud computing avec EC2. Cette technologie AWS EC2 est à ce jour devenue obsolète et ne sert que pour l'hébergement dans le cloud. Si nous devions perdre la base de P° et l'instance CMDB (glpi) présente dans AWS nous pourrions toujours basculer vers le serveur de secours on-premise qui lui est configuré afin d'offrir une réplication circulaire.

D'après moi, ce choix initial ne convient pas à une exploitation en HA qui nécessite aucune interruption de service et doit s'affranchir d'une re-synchroniation avec la dB AWS ou celle de secours en cas de vrai crash.

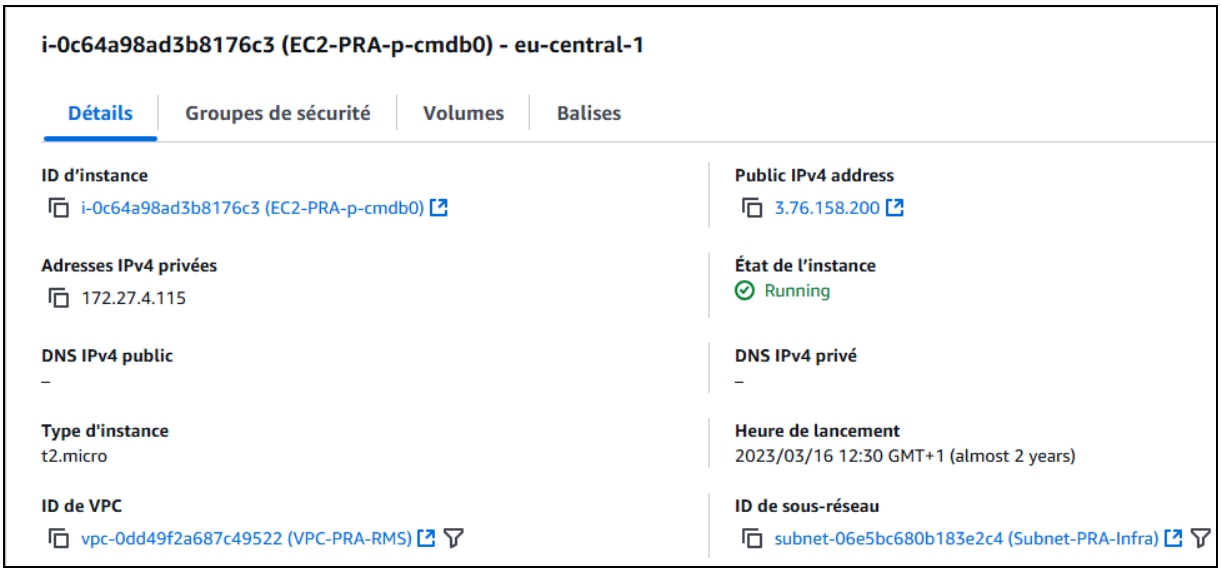
Plusieurs modèles de réplication ont été étudiés (RDS, DMS avec Aurora ...). De mon côté j'ai retenu la technologie AWS Aurora avec DMS qui permettraient une réplication en temps réel avec des temps de chargement considérablement diminués (technologie DMS). « AWS Aurora est conçu pour offrir des performances élevées et une grande disponibilité, combinant la rapidité et la fiabilité des bases de données commerciales haut de gamme avec la simplicité et le coût réduit des bases de données open-source »

Tout au long de ce « parcours MCO » j'ai appris à reconstruire les objets en m'isolant de toute adhérence avec la P°. J'ai choisi le réseau VPC disponible, associé les régles de sécurités, construit les clé de connexions (RSA) , initialisé des points de terminaison à partir de briques AWS Cloudformation afin de pour pouvoir simuler depuis AWS DMS une réplication temps réel entre deux base de données SQL situées dans le cloud.

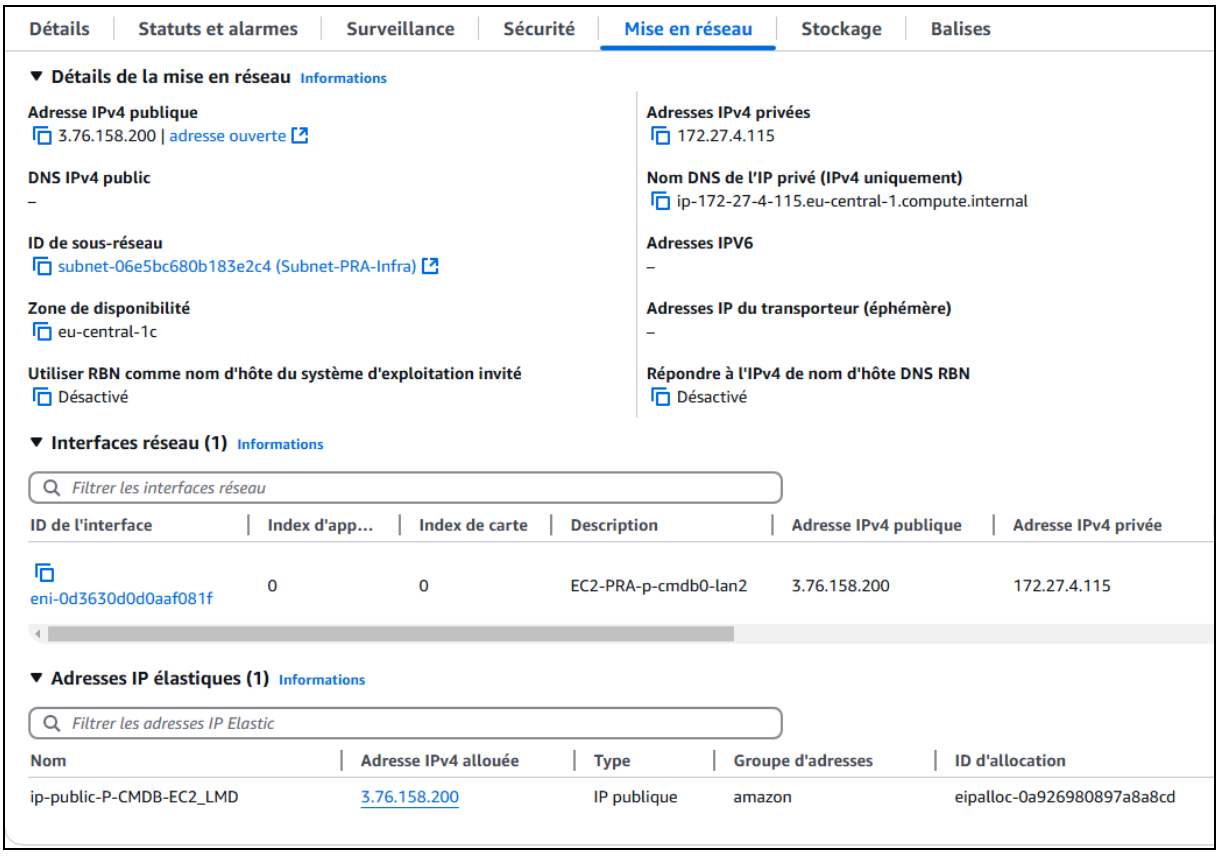
Il faut extrapoler ce type de processus migratoire et de réplication pour d'autres usages comme une réplication PostgreSQL ou Oracle avec le service RDS. Cette technologie est préconisable lors de changement de paradigme par les clients actuels ou à venir et ceci sans tenir compte de leur taille. AWS gère toute l'infrastructure et les DBA ne devraient se concentrer que sur les phases de réplication et d'optimisation des transferts de données vers le cloud.

Analyse de l'existant

L'hébergement actuel dans AWS et celui présenté dans la vue ci-dessous et l'@IP privé pointe aujourd'hui sur la VM **EC2-PRA-p-cmdb0 :** @IPv4 **172.27.4.115** (voir également en annexe l'architecture AWS et les connexions opérées avec le serveur local depuis ce VPC)



Cette VM **EC2-PRA-p-cmdb0 :** @IPv4 **172.27.4.115** est accessible depuis une @IP public qui est un @IP élastique : ip-public-P-CMDB-EC2_LMD (@ip publique persistante)



Le script de PRA on premise n'est plus activé à partir de la VM hébergée sur la PFS : GRE_MAIN et ceci depuis le **05-01-2024** (*) . En effet en date du 03-01-2024 SPIE ICS a réalisé un arrêt électrique et un incident notable est survenu par la suite avec perte de tables dans la base glpi. Côté DBA nous avons palier à cet incident en réinjectant les DATA et privilégié depuis cette date le mode de réplication circulaire entre le serveur de P° et le serveur de secours. Nous avons cependant maintenu le mode uni-directionnel pour les MAJ entre le serveur de P° (master) et le serveur PRA (slave hébergé sur AWS). Pour autant la synchronisation s'opère bien entre les 3 nodes et le node PRA est priorisé pour les MAJ à l'instar du node de secours. En résumé en cas d'incident majeur sur la P° il est toujours possible de se reconnecter sur le serveur de secours ou le serveur PRA ce qui constitue une double sécurité (voir en annexe). Cependant nos actions auront conduit à une interruption de service pour le CSM.

3 type de réplication sont possibles :

1. Réplication circlulaire (master<->master) : CMDB <-> S-CMDB
2. Réplication uni-directionnelle avec synchronisation (master->slave) : CMDB -> PCMDB
3. Réplication uni-directionnelle inversée (slave devenant le master)

C'est la raison pour laquelle je me suis interressé plus récemment à une autre solution permettant cette fois de limiter les interruptions de services. Cette solution reste avant tout transposable à d'autres technologies que MySQL (PostgreSQL, Oracle , MSSQL ...). Dès lors AWS propose d'utiliser son service Aurora avec DMS pour garantir une réplication temps réel.

Certains pré-requis côté infrastructure sont à prendre en compte et c'est ce qui sera exposé dans le prochain chapître.

(*) Avant cette date nous avions constaté de très fortes latences obligeant à réaliser ces actions quotidienne de synchronisation en HNO. La synchronisation s'opèrait à partir de la cmd (mysqldump) vers un fichier au format archive qui était ensuite copié vers le serveur de destination pour y être rechargé (...) Pour configurer le transfert il fallait interrompre l'activité le temps de pouvoir effectuer la resynchronisation. Cependant toutes ces automatisations nécessitaient des coupures de services en Production.

Pré-requis DBA

Nouvelle infrastructure VPC

Je me suis appuyé sur le VPC `vpc-cb8c0ba0` qui a servi au préalable à plusieurs labs (...)
La configuration modifiée après le rajout de mon côté de l'infrastructure **RDS** devient celle-ci :

```
~ $ aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-cb8c0ba0" --query
"Subnets[*].{ID:SubnetId,AZ:AvailabilityZone,CIDR:CidrBlock}" --output table

-------------------------------------------------------------------
|                        DescribeSubnets                          |
+---------------+--------------------+----------------------------+
|      AZ       |       CIDR         |            ID              |
+---------------+--------------------+----------------------------+
|  eu-central-1c|  172.31.0.0/20     |  subnet-02e07e4f           |
|  eu-central-1b|  172.31.48.0/25    |  subnet-022a66bb39d4e9bba  |
|  eu-central-1a|  172.31.16.0/20    |  subnet-a21958c9           |
|  eu-central-1b|  172.31.32.0/20    |  subnet-837ac4fe           |
|  eu-central-1c|  172.31.48.128/25  |  subnet-0d0bf290cc6558356  |
|  eu-central-1a|  172.31.49.0/25    |  subnet-09d17ca70f535db88  |
+---------------+--------------------+----------------------------+
```

On note 3 zones de disponibilité sur ce VPC comprenant chacun un sous-réseau public et un sous réseau privé (voir le diagramme associé).

Pour différencier un sous-réseau public (accès depuis Internet) d'un sous-réseau privé qui lui n'est accessible que dans le cloud AWS, il faut vérifier la présence ou pas d'une gateway :

```
~ $ aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-cb8c0ba0" --
query "RouteTables[*].{ID:RouteTableId,Routes:Routes}" --output text
```

Liste des tables routes

```
rtb-0a285bced2b8151c2
ROUTES  172.31.0.0/16   local   CreateRouteTable        active
rtb-31143c5a
ROUTES  172.31.0.0/16   local   CreateRouteTable        active
ROUTES  0.0.0.0/0       igw-96d9b0fe    CreateRoute     active
```

Sous-réseaux concernés pour la `rtb-31143c5a` pour le VPC `vpc-cb8c0ba0 (région Francfort)`

~ $ aws ec2 describe-route-tables --route-table-ids rtb-31143c5a --query "RouteTables[*].Associations[*].SubnetId" --output table

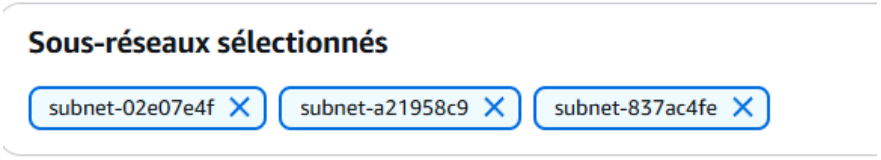
---

|DescribeRouteTables| -> Aucune remontée !!

~ $ aws ec2 describe-route-tables --route-table-ids rtb-0a285bced2b8151c2 --query "RouteTables[*].Associations[*].SubnetId" --output table

```
-------------------------------------------------------------------------------
|                              DescribeRouteTables
+--------------------------+----------------------------+--------------------------
|  subnet-0d0bf290cc6558356|  subnet-09d17ca70f535db88  |  subnet-022a66bb39d4e9bba
+--------------------------+----------------------------+--------------------------
```

En résumé on trouve 3 sous-réseau associés avec la rtb-0a285bced2b8151c2 mais aucun sous réseau assocvié avec la rtb-31143c5a ! Ce qu'il faut corriger pour pouvoir allouer la GW **`igw-96d9b0fe`** et se connecter depuis internet. Pour cela j'ai associé les 3 sous-réseau à la route table comme renseigné dans le diagramme :



Nouvelle infrastructure EC2

J'ai utilisé le sous-réseau public pour créer le jump host EC2-POP-MySQL qui reste accessible avec l'@IPv4 publique 3.125.48.119 et depuis internet avec le DNS ec2-3-125-48-119.eu-central-1.compute.amazonaws.com. Après l'ajout de cette instance EC2 `i-0fe7b8b1fca2fa838` la configuration devient celle-ci :

```
~ $ aws ec2 describe-instances --filters "Name=vpc-id,Values=vpc-cb8c0ba0" --query
"Reservations[*].Instances[*].{ID:InstanceId,Type:InstanceType,State:State.Name}" -
-output table

-------------------------------------------------
|               DescribeInstances               |
+----------------------+----------+-------------+
|          ID          |  State   |    Type     |
+----------------------+----------+-------------+
|  i-0fe7b8b1fca2fa838 |  running |  t2.micro   |
+----------------------+----------+-------------+
```

On note un configuration minimaliste pour des raisons de $

Les groupes de sécurité (SG) applicable (à date) à cette instance EC2 jump host (public) sont :

```
~ $ aws ec2 describe-security-groups --filters "Name=vpc-id,Values=vpc-cb8c0ba0" --
query "SecurityGroups[*].{ID:GroupId,Name:GroupName,Description:Description}" --
output table
```

```
|  Security group attached to mysql-rds-db to allow EC2 instances with specific security groups attached to connect to the database.
Modification could lead to connection loss. |  sg-07b4114cf5e1ba108 |  rds-ec2-1                                                    |

|  Security group attached to mysqldb to allow EC2 instances with specific security groups attached to connect to the database.
Modification could lead to connection loss.      |  sg-071f8a9e53527df19 |  rds-ec2-2                                                  |

|  default VPC security group
|  sg-17e0547a          |  default                                          |

|  Security group attached to instances to securely connect to mysql-rds-db. Modification could lead to connection loss.
|  sg-061bc86cbef3d4f16 |  ec2-rds-1                                        |

|  Security group attached to instances to allow them to securely connect to mysqldb. Modification could lead to connection loss.
|  sg-036d5a2b0caecbcbf |  ec2-rds-2                                        |

|  DB Security group to access MySQL database
|  sg-04a94a368d04a52f8 |  Pile-Mariadb-POC-B-DBSecurityGroup-H4lwMKI7F7Df  |
```

Vous noterez la présence du SG associé à la « Pile-Mariadb-POC-B … » qui sera développé dans ce document et sert de poit de terminaison pour la brique DMS. J'ai connecté l'instance RDS mysqldb avec l'EC2-POC-MySQL

Enfin j'ai adapté chaque règle E/S à chacun des SG pour l'instance EC2-POC-MySQL
En priorisant les protocoles tcp 22 et 3306 pour un accès SSH à EC2 et à l'instance MySQL
Il est toujours possible de rajouter des règles réseau pour E/S (**INBOUNDRULES/OUTBOUNDRULES**)

Un règle dans un SG peut rajouter ou enlever une contrainte par rapport à un autre SG. Ici pour se connecter en entrée à EC2 il faut SSH port 22 et pour se connecter en sortie à la dB MySQL le port 3306. Ces 2 règles sont assumées dans le SG : **sg-061bc86cbef3d4f16** *et* **sg-07b4114cf5e1ba108**

```
~ $ aws ec2 describe-security-groups --group-ids <SG> --query
"SecurityGroups[*].{ID:GroupId,InboundRules:IpPermissions,OutboundRules:IpPermissio
nsEgress}" --output text
```

```
sg-07b4114cf5e1ba108
INBOUNDRULES    22      tcp     22
IPRANGES        0.0.0.0/0       test -> non secure et valable que pour ce test
INBOUNDRULES    3306    tcp     3306
USERIDGROUPPAIRS        Rule to allow connections from EC2 instances with sg-
061bc86cbef3d4f16 attached sg-061bc86cbef3d4f16    216285719826
```

```
sg-071f8a9e53527df19
INBOUNDRULES    3306    tcp     3306
USERIDGROUPPAIRS        Rule to allow connections from EC2 instances with sg-
036d5a2b0caecbcbf attached sg-036d5a2b0caecbcbf    216285719826
```

```
sg-17e0547a
INBOUNDRULES    -1
USERIDGROUPPAIRS        sg-17e0547a     216285719826
OUTBOUNDRULES   -1
IPRANGES        0.0.0.0/0
```

```
sg-061bc86cbef3d4f16
INBOUNDRULES    22      tcp     22
IPRANGES        3.125.48.119/32 -> @IP public EC2-POC-MySQL
OUTBOUNDRULES   3306    tcp     3306
USERIDGROUPPAIRS        Rule to allow connections to mysql-rds-db from any
instances this security group is attached to sg-07b4114cf5e1ba108    216285719826
```

```
sg-036d5a2b0caecbcbf
OUTBOUNDRULES   3306    tcp     3306
USERIDGROUPPAIRS        Rule to allow connections to mysqldb from any instances
this security group is attached to      sg-071f8a9e53527df19    216285719826
```

```
sg-04a94a368d04a52f8
INBOUNDRULES    3306    tcp     3306
IPRANGES        10.0.0.0/24
OUTBOUNDRULES   -1
IPRANGES        0.0.0.0/0
```

En construisant la nouvelle instance jump host EC2-POC-MySQL, AWS m'a alloué automatiquement la keypair « ppk_TEST » dont je ne suis pas à l'origine de sa création et pour laquelle je n'ai pas l'emplacement en local de la ppk/pem. Pour palier à cette situation j'ai dû réaliser une opération de réallocation d'une nouvelle clé. Pour cela j'ai dû reconstruire une instance EC2 avec une AMI et supprimer l'ancienne pour ensuite la renommer EC2-POC-MySQL. *On ne peut pas sur une EC2 active modifier la keypair sans réaliser ce type de « recovery »* :

Construction de la nouvelle clé :

```
~ $ aws ec2 create-key-pair --key-name JGDKeyPair --query 'KeyMaterial' --output text > JGDKeyPair.pem
```

Positionnement des permissions pour la clé :

```
~ $ aws chmod 400 JGDKeyPair.pem
```

Arrêt de l'intance EC2-POC-MySQL

```
~ $ aws ec2 stop-instances --instance-ids i-0fe7b8b1fca2fa838
~ $ aws ec2 wait instance-stopped --instance-ids i-0fe7b8b1fca2fa838
```

Création de la nouvelle instance EC2-POC-MySQL-B avec AMI à l'image de EC2-POC-MySQL

```
~ $ aws ec2 create-image --instance-id i-0fe7b8b1fca2fa838 --name " EC2-POC-MySQL-B " --no-reboot
```

!! cette construction AMI peut échouer avec le message ci-dessous si un snapshot été en cours d'exécution 🤔

```
"An error occurred (IncorrectState) when calling the RunInstances operation: Snapshot is in invalid state – pending" !!
```

Il faut attendre quelques minutes avant que le snapshot ne soit terminé. (voir annexe)

Lancement de la nouvelle instance avec la nouvelle clé :

```
~ $ aws ec2 run-instances --image-id ami-076bed9bdef092f28 --count 1 --instance-type t2.micro --key-name JGDKeyPair --subnet-id subnet-837ac4fe
```

Résiliation de l'ancienne instance EC2-POC-MySQL (optionnel)

```
~ $ aws ec2 terminate-instances --instance-ids i-0fe7b8b1fca2fa838
```

Dans notre cas l'EC2-POC-MySQL apparaît en grisé dans le diagramme

Configuration de l'infrastructure RDS

Prérequis

Pour pouvoir se connecter depuis **AWS CLI** il m'a fallu construire une key-pair (ppk Putty)

➔ **my-key-ec2-jgd.pem**

J'ai ensuite tenté d'initialiséer une db à partir d'une pile sous **Cloudformation** (IAC). Pour cela je me suis appuyé sur un fichier modèle en Yaml (template) qui au téléchargement est stocké dans un buckset **S3** :

➔ **Mariadb_CF.yaml**

La génération de la pile donne naissance à un « jeu » qui comprend la conf de la pile **RDS** et une fois exécuté retrourne les incidents sur paramètrage -> exemple la release mariadb n'est pas compatible ! AWS invite à vérifier la matrice de compatibilité ce qui est un prérequis.
MariaDB on Amazon RDS versions - Amazon Relational Database Service

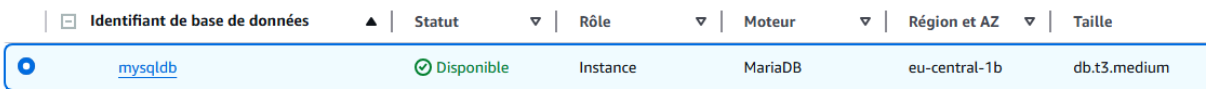
Cependant il faut noter que les versions mises à disposition dépendent d'une région à l'autre et c'est la raison pour laquelle il reste préférable de bien vérifier les release mise à disposition par AWS dans sa région (ici Francfort). En utilisant **AWS CLI** on obtient la liste exhaustive :

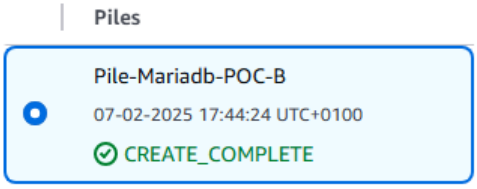
➔ aws rds describe-db-engine-versions --engine mariadb --query "*[].{Engine:Engine,EngineVersion:EngineVersion}" --output text

J'ai donc choisi ici la dernière version 10.4.32 disponible sur la région de Francfort et RAZ le template **MariadB_CF.yaml** en le renommant **MariaDB10.4.32_CF.yaml** pour rejouer cette pile**.** En prérequis, il faut remplir exhaustivement ou compléter les rubriques yaml du template (voir le nouveau jeu de pile en annexe) Un prérequis consiste à vérifier db-engine avec AWS CLI :

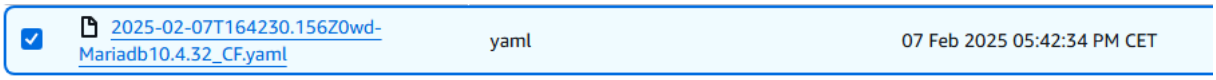
➔ aws rds describe-db-engine-versions --query "DBEngineVersions[].DBParameterGroupFamily"

J'ai donc construit à partir de la pile de **Cloudformation**, un base **RDS** que j'ai nommé mysqldb avec la release mariadb.10.4.32 (voir annexe)





🖐 petit conseil pensez svp à récupérer la pile ou la versionner sur S3 pour un usage ultérieur



De plus il faut prévoir d'installer sur la nouvelle EC2 le client mariadb … Or force est de constater qu'AWS a provisionné un instance linux t2.micro dont le dépôt ne contient pas un repository pour mariadb avec la version OS fedora 🤔 alors que ce dépôt est présent par défaut sur les autres OS :

MariaDB Package Repository.

The distributions currently supported by the script include:

- Red Hat Enterprise Linux (RHEL) 8, and 9
- Debian 10 (Buster), 11 (Bullseye), 12 (Bookworm)
- Ubuntu 20.04 LTS (Focal), 22.04 LTS (Jammy), and 24.04 LTS (Noble)
- SUSE Linux Enterprise Server (SLES) 12 and 15

Version OS de l'EC2

```
[ec2-user@ip-172-31-35-228 ~]$ cat /etc/*release*
Amazon Linux release 2023.6.20250203 (Amazon Linux)
cpe:2.3:o:amazon:amazon_linux:2023
NAME="Amazon Linux"
VERSION="2023"
ID="amzn"
ID_LIKE="fedora"
VERSION_ID="2023"
PLATFORM_ID="platform:al2023"
PRETTY_NAME="Amazon Linux 2023.6.20250203"
```

Il faut donc trouver une alternative pour pouvoir déposer les bons packages avant de poursuivre l'installation de mariadb en vérifiant sur le site officiel de MariaDB :

```
# MariaDB 10.6 Fedora repository list - created 2025-02-09 16:56 UTC
# https://mariadb.org/download/
[mariadb]
name = MariaDB
# baseurl = https://rpm.mariadb.org/10.6/fedora/$releasever/$basearch
baseurl = https://mirror.mva-n.net/mariadb/yum/10.6/fedora/$releasever/$basearch
# gpgkey= https://rpm.mariadb.org/RPM-GPG-KEY-MariaDB
gpgkey=https://mirror.mva-n.net/mariadb/yum/RPM-GPG-KEY-MariaDB
gpgcheck=1
```

Echec de l'install avec Fedora concernant les release non supportées !

J'ai donc pris le choix de créer une Instance RedHat : **EC2-POC-MariaDB** : @IP 172.31.14.194
https://mariadb.org/download/?t=repo-config&d=Red+Hat+Enterprise+Linux+9&v=11.4&r_m=mva

```
# MariaDB 11.4 RedHatEnterpriseLinux repository list - created 2025-02-09 18:13 UTC
# https://mariadb.org/download/
[mariadb]
name = MariaDB
# rpm.mariadb.org is a dynamic mirror if your preferred mirror goes offline. See https:
# baseurl = https://rpm.mariadb.org/11.4/rhel/$releasever/$basearch
baseurl = https://mirror.mva-n.net/mariadb/yum/11.4/rhel/$releasever/$basearch
# gpgkey = https://rpm.mariadb.org/RPM-GPG-KEY-MariaDB
gpgkey = https://mirror.mva-n.net/mariadb/yum/RPM-GPG-KEY-MariaDB
gpgcheck = 1
```

(…)

Une fois le pack mariadb déployé … vous pouvez tester votre connexion depuis EC2-POC-MariaDB et vous connecter depuis le point de terminaison (endpoint) à l'instance dB mysqldb hébergée sur le sous-réseau `subnet-a21958c9` initialisée avec la brique `MariaDB_CF.yaml` de **AWS Cloudformation**, comme indiqué dans le diagramme : :

```
[ec2-user@ip-172-31-14-194 ~]$ sudo mariadb -h mysqldb.cd1xhslmisxu.eu-central-
1.rds.amazonaws.com -P 3306 -u master -D mysqldb -p
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3174
Server version: 10.4.34-MariaDB-log Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

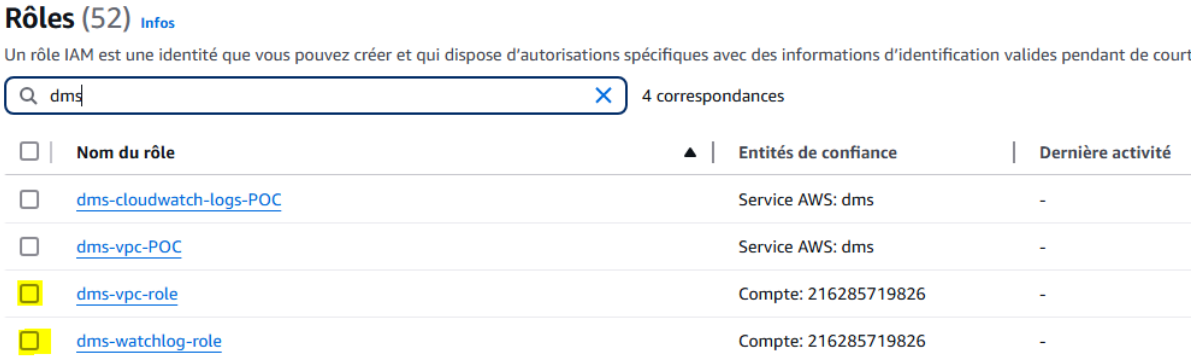
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [mysqldb]> show tables;
+-------------------+
| Tables_in_mysqldb |
+-------------------+
| orders            |
| products          |
| users             |
+-------------------+
3 rows in set (0.001 sec)

MariaDB [mysqldb]> SELECT DISTINCT TABLE_NAME, INDEX_NAME,NON_UNIQUE FROM
INFORMATION_SCHEMA.STATISTICS WHERE TABLE_SCHEMA = 'mysqldb' and INDEX_NAME <>
'PRIMARY';
+------------+------------+------------+
| TABLE_NAME | INDEX_NAME | NON_UNIQUE |
+------------+------------+------------+
| orders     | user_id    |          1 |
+------------+------------+------------+
1 row in set (0.002 sec)

MariaDB [mysqldb]>
```

Pour pouvoir utiliser la technologie DMS il faut au préalable disposer des droits IAM en autorisant AWS à réaliser des actions DMS dans un VPC et pouvoir activer le service CloudWatchLogs. L'entité de confiance qui assurera ce rôle pour vous reste votre compte (voir en annexe la mise en place avec AW CLI).



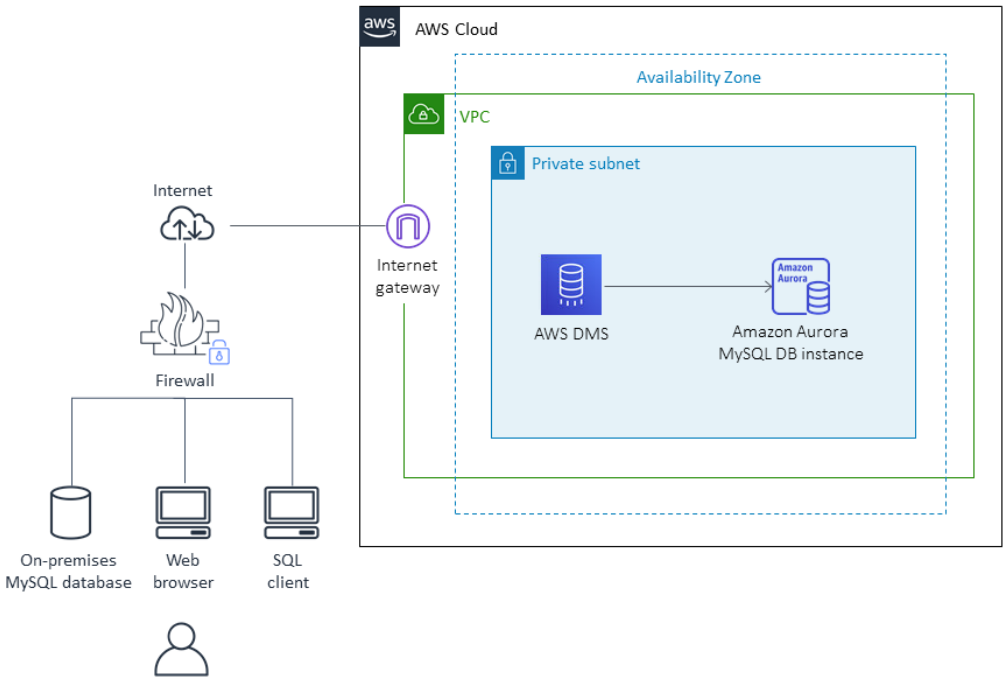


Migration – réplication avec DMS

Architecture proposée

Architecture de migration des données

En utilisant AWS DMS :



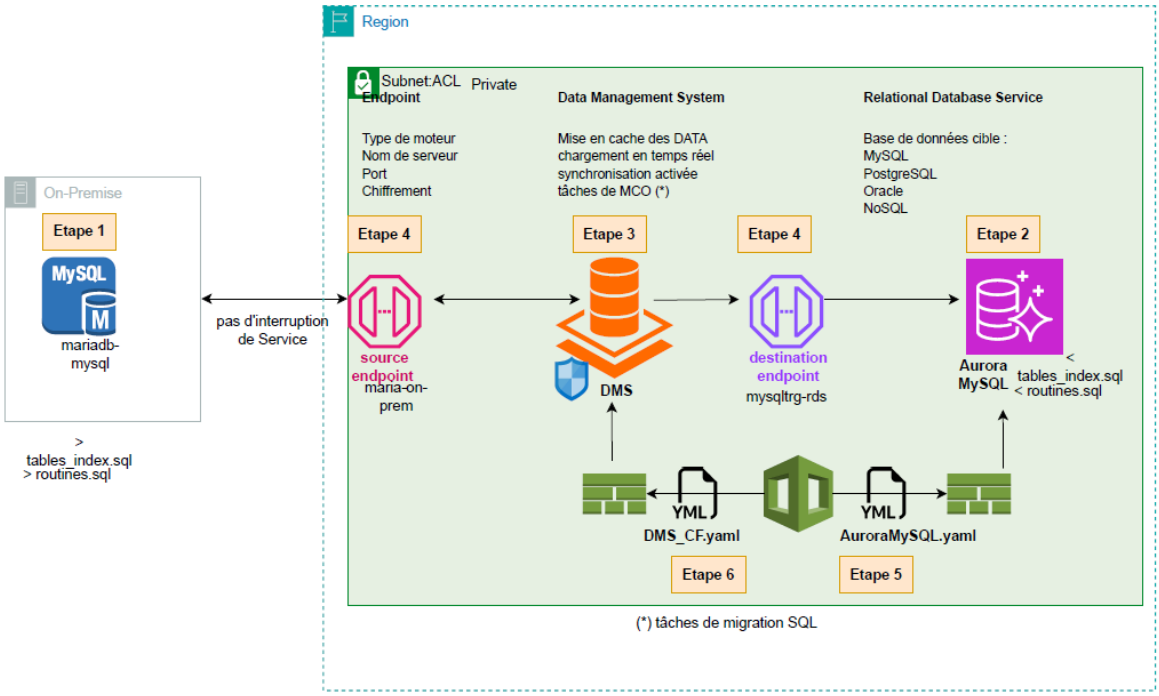
Installation d'Aurora MySQL comme dB cible

Construction de la stack à partir du fichier yaml

AWS permet de provisionner un service **Aurora MySQL** à partir de la construction d'une brique à partir du chargement d'un fichier de configuration codé en yaml. Le fichier de configuration est AuroraMysql_CF.yaml dont le fichier se trouve sur un stockage « public » S3 et vous trouverez le lien de chargement ci-dessous :

https://aws-database-blog.s3.amazonaws.com/artifacts/mariadb-to-aurora-mysql-migration/AuroraMysql_CF.yaml

il faut utiliser **AWS CloudFormation** pour exploiter cette brique yaml (Etape 5)



Une fois la brique yaml chargée il faut spécifier chaque paramètre avant de créer la tâche (voir en annexe)

Vous devez importer la 1ère fois les structures des tables et des index afin de répliquer à ISO périmètre la dB MySQL hébergée par Aurora avec la dB MySQL on premise. Ensuite une synchronisation temps réel sera mise en place à partir de tâches du service **AWS DMS**.

Nb : l'import des structures de tables et d'index s'opère à partir du dump (mysqldump) obtenu depuis la dB MySQL on premise :

```
$ mysqldump --no-data --no-create-db --single_transaction -u root -p <db_Name> --skip-triggers > mysql_tables_indexes.sql

$ mysqldump -u root --routines --no-create-info --no-data --no-create-db --skip-opt -p <db_Name> > routines.sql
```

Import depuis AWS Aurora (dB cible) :

```
$ mysql -h <ARN>  -u master -p <db_Name> -P 3306 < mysql_tables_indexes.sql
```

Nb : avant l'import des routines avec le script routines.sql, il vous faut provisionner l'instance DMS qui est l'interface AWS pour exécuter et planifier des tâches de réplication

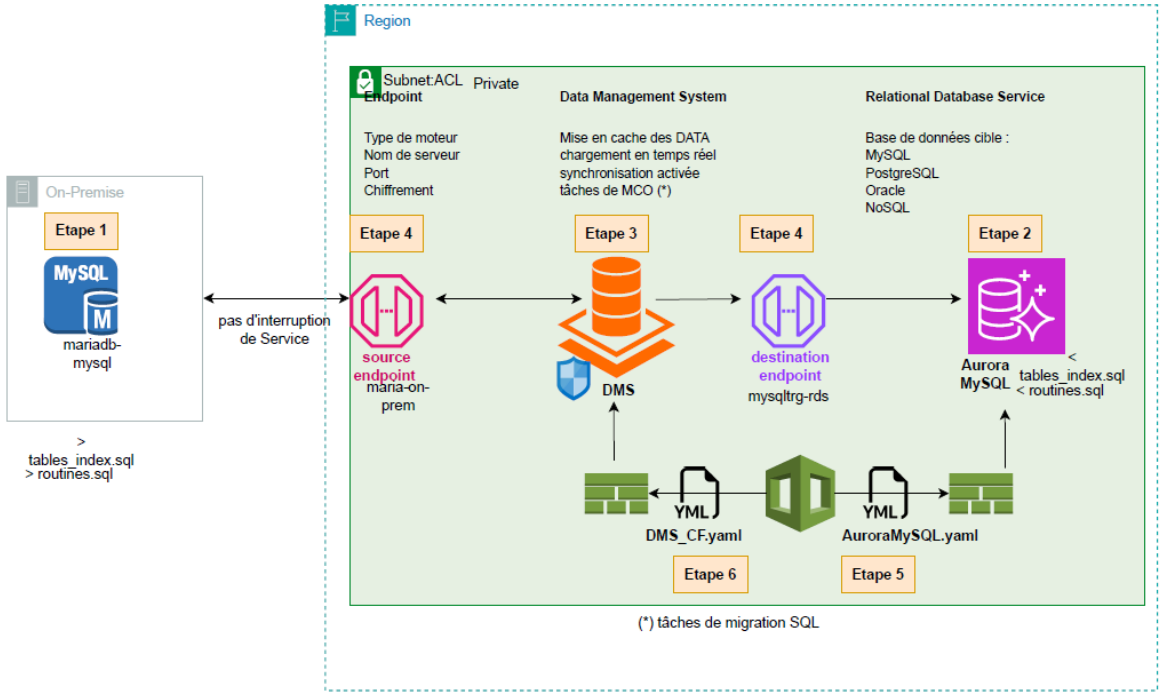
Installation de l'instance AWS DMS pour la réplication

Construction de la stack à partir du fichier yaml

AWS permet de provisionner une instance **AWS DMS** à partir de la construction d'une brique depuis une configuration yaml. Le fichier de configuration est DMS_CF.yaml. Ce fichier se trouve sur un stockage « public » S3 et vous trouverez le lien de chargement ci-dessous :

https://aws-database-blog.s3.amazonaws.com/artifacts/mariadb-to-aurora-mysql-migration/DMS_CF.yaml

il vous faut utiliser **AWS CloudFormation** pour exploiter cette brique yaml (Etape 6)



Une fois la brique yaml chargée il faut spécifier chaque paramètre avant de créer la tâche (voir en annexe)

La base Aurora MySQL construite s'appelle mariadb-mysql le point de terminaison source (source endpoint) s'appelle maria-on-prem et le point de terminaison cible (target endpoint) s'appelle mysqltrg-rds. Ce point de terminaison cible offre une connection qui permet de réaliser les duplication avec AWS DMS en s'affranchissant des foreign key (clé étrangère) ce qui permet des chargement sans contraintes :

ExtraConnectionAttributes : "initstmt=SET FOREIGN_KEY_CHECKS=0;parallelLoadThreads=1"

Avant de construire la tâche qui va permettre de mettre en place un modèle de réplication planifié avec un très faible latence lors des chargements de DATA, il vous faut tester au préalable le bon fonctionnement des connexions source maria-on-prem et cible mysqltrg-rds.

Création de tâches AWS DMS

Ce travail est réalisé à partir du service **AWS Database Migration Tasks**

La création de la tâche de réplication s'appuie sur un template que vous trouverez en annexe.
Ici il faut privilégier comme type de réplication de données : « Migrate existing data and replicate changes for CDC ». Dans le contexte d'AWS DMS, "CDC" signifie Change Data Capture. C'est une technique qui permet de suivre et de capturer les changements effectués sur une base de données afin de les répliquer vers une autre base de données (voir annexe)

Validation du réplica AWS DMS

Cette étape permet de valider le fait que les DATA ont bien été transférées de la source vers la cible. Une table des statistiques DML permet d'avoir une vue exhaustive des changements qui se sont opérés (INSERT, DELETE,UPDATE, ...) : voir en annexe le type de table obtenue.

Cut over

Au niveau de cette étape il faut activer les routines (procédures, fonctions, triggers ...) en utilisant les commandes ci-dessous :

```
$ mysql -h <ARN>  -u master -p <dB_Name> -P 3306 < routines.sql
```

A partir de l'instant ou les vues, les procédures et les fonctions sont peu modifiées, l'activation des routines reste une « formalité » pour reconditionner la nouvelle instance de base de données hébergées sur MariadB.

Ceci reste optionnelle mais une fois les routines chargées dans la nouvelle base Aurora, il est toujours possible de réaliser une connexion (mysql) pour extraire toutes les informations depuis les vues information_schema.<routines|triggers> de la base migrée et comparer avec la source dB MySQL on premise.

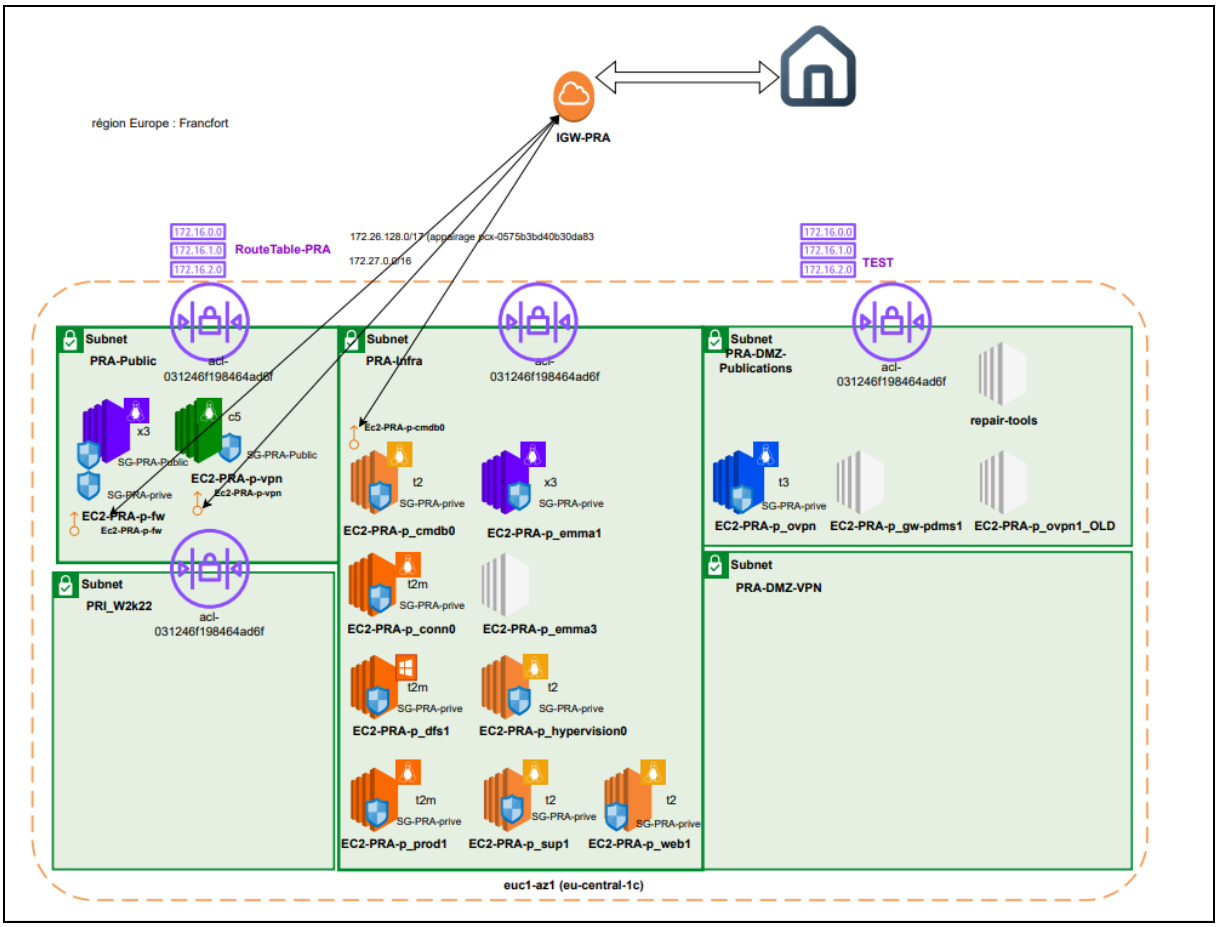
Annexes

Architecture du VPC PRA-RMS (AWS)

La connexion avec l'extérieur est obtenue à partir de la gateway (IGW-PRA) pour un accès vers internet et de la connexion d'apparaige (pcx-0575b3bd40b30da83) pour un accès vers un autre VPC. Ces 2 composants sont routés à travers une table de routage (RouteTable-PRA). Il est à noter que cette connexion d'apparaige est une connexion entre 2 VPC qui permet d'acheminer le traffic entre eux de manière privée. Cela permet de faire communiquer les instances EC2 entre elles.

VPC DEMANDEUR : **PFS-VPC** (vpc-0fb5f15e3a3369487) construit par l'Equipe 3IO (tag)

VPC ACCEPTEUR : **VPC-PRA-RMS** (vpc-0dd49f2a687c49522) ci-dessous



On notera cependant des potentielles « failles de sécurité » sur l'actuel sous **réseau PRA-Infra** même si cela devait au départ ne concerner que les flux internes entre les différents sous-réseau du VPC … ce n'est cependant pas le cas. En effet pour que ce sous-réseau soit « privé » il faudrait qu'il n'y ait aucun accès vers l'extérieur depuis une gateway par exemple (*). Or force est de constater que ce n'est pas ici le cas :

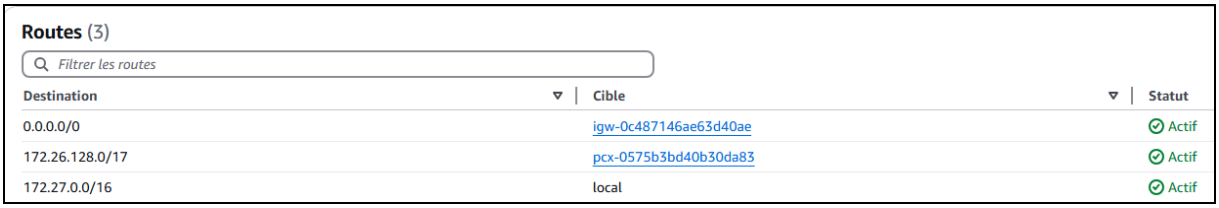


De plus et concernant le Groupe de sécurité, j'ai pu noter un côté plutôt « permissif » (*) pour l'usage des 3 protocoles (ICMS, TCP et UDP) et cela concerne toutes les ranges IPv4 (idem pour IPv6) :

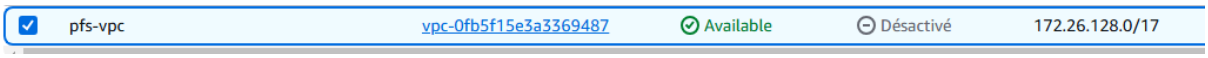
ci-dessous les règles entrantes du Groupe de sécurité (SG-PRA-prive) :

☐	–	sgr-098a10b7c64d4264b	IPv4	Tous les ICMP - IPv4	ICMP	Tous	0.0.0.0/0
☐	–	sgr-070169ec4041b3a20	IPv4	Tous les TCP	TCP	0 - 65535	0.0.0.0/0
☐	–	sgr-026fec95e6b69fb62	IPv4	Tous les UDP	UDP	0 - 65535	0.0.0.0/0

Maintenant si on s'interrese au **réseau PRA-Public** qui est exposé avec l'extérieur et donc internet et qui s'appuie sur la passerelle (IGW-PRA) on trouve :



L'apparaige avec l'IPv4 172.26.128.0 est réalisé avec un autre VPC (pfs-vpc) situé dans une autre région (Paris) qui correspondrait plus à un Bastion AWS mis en place par l'Equipe 3IO :



(*) Par définition un sous-réseau est dit privé lorsqu'il n'a aucune route IP vers l'extérieur. La présence d'une passerelle vers l'extérieur signifie qu'il est public. Seul un accès internet via une NAT est possible.

(**) Il peut effectiment s'agir de la mise en place d'un 1er lab par l'anciene Equipe d'Infrastructure qui ensuite a été converti en environnement de P°. Mais AWS préconise des restrictions. Un Groupe de Sécurité est considéré comme *satefull* c'est-à-dire que tout ce qui n'est pas autorisé est interdit. Or ici tout est permis en matière d'accès interne VPC !

Réplication circulaire avec MySQL

Depuis CITRIX force est de constater que le CSM dispose désormais et depuis janvier 2024 d'une double sécurité permettant à tout moment et lors de crash de la PFS de basculer sur le PRA (AWS). A défaut si le crash devait concerner la VM de P° et celle du PRA qui exploite la CMDB, il est toujours possible de basculer les connexions du CSM sur le serveur VM de secours.

Les 3 DNS pointent vers les serveurs glpi dont les DATA sont aujourd'hui à ISO périmètre :

DNS avec la connexion **CMB** : https://cmdb.gre.loc

DNS avec la connexion **S-CMDB** : https://s-cmdb.gre.loc

DNS avec la connexion **P-CMDB** : https://p-cmdb.gre.loc (AWS)

Template AWS CloudFormation pour le déploiement d'AWS Aurora

Stack name — Enter a name for the stack.

CIDR — Enter the CIDR IP range to access the instance.

DBBackupRetentionPeriod — The number of days for backup retention.

DBInstanceClass — Enter the instance type of the database server.

DBMasterPassword — Enter the master password for the DB instance.

DBMasterUsername — Enter the master user name for the DB instance.

DBName — Enter the name of the database.

DBSubnetGroup — Enter the DB subnet group.

Engine — Enter the Aurora engine version; the default is 5.7.mysql-aurora.2.03.4.

PreferredBackupWindow — Enter the daily time range in UTC during which you want to create automated backups.

PreferredMaintenanceWindow — Enter the weekly time range in UTC during which system maintenance can occur.

VPCID — Enter the ID for the VPC to launch your DB instance in.

Template AWS CloudFormation pour le déploiement de l'instance DMS

Stack name — Enter a name for the stack.

AllocatedStorageSize — Enter the storage size in GB. The default is 200 GB.

DMSReplicationSubnetGroup — Enter the subnet group for DMS replication.

DMSSecurityGroup — Enter the security group for DMS replication.

InstanceType — Enter the instance type.

SourceDBPort — Enter the source database port.

SourceDatabaseName — Enter the source database name.

SourceServerName — Enter the IP address of the source database server.

SourceUsername — Enter the source database user name.

SourcePassword — Enter the source database password.

TargetDBPort — Enter the target database port.

TargetDatabaseName — Enter the target database name.

TargetServerName — Enter the IP address of the target database server.

TargetUsername — Enter the target database user name.

TargetPassword — Enter the target database password.

Fonctionnalités pour une migration CDC

Fonctionnalités de CDC :

Capture des Modifications : CDC enregistre les insertions, mises à jour et suppressions effectuées sur la base de données source.

Réplication en Temps Réel : Les changements capturés peuvent être appliqués en temps réel ou presque en temps réel à la base de données cible, permettant une synchronisation continue.

Scénarios d'Utilisation : Idéal pour les migrations de bases de données où l'on souhaite transférer des données existantes tout en continuant à répliquer les modifications qui se produisent pendant la migration.

Avantages du CDC : Minimisation du Temps d'Inactivité : Permet de migrer sans arrêter les opérations. Synchronisation Continue : Assure que la base de données cible reste à jour avec les modifications effectuées sur la source.

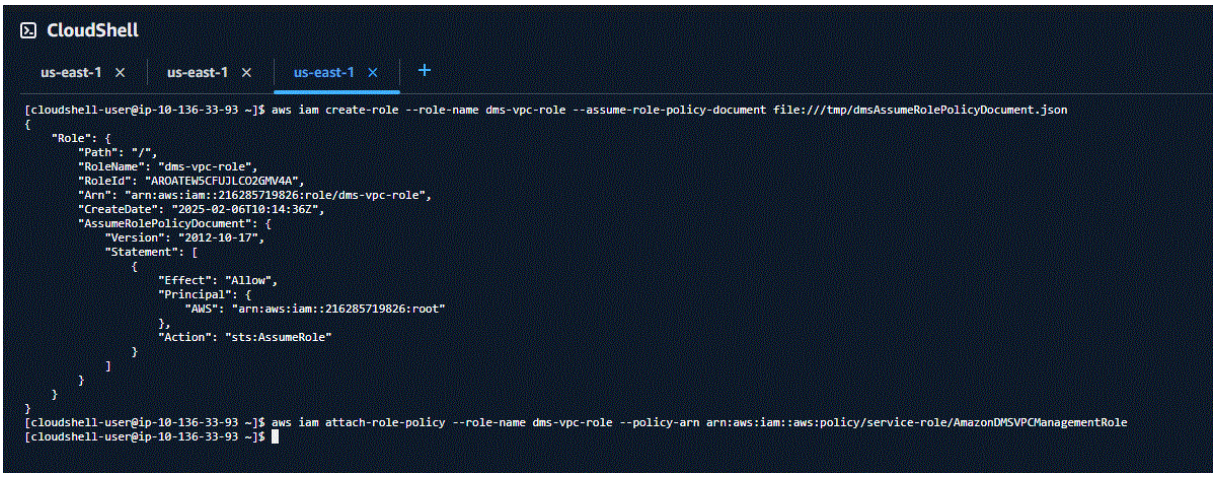
En résumé, CDC est essentiel pour les scénarios où il est crucial de maintenir la cohérence des données entre les systèmes source et cible pendant et après une migration.

Build IAM role(s) pour DMS

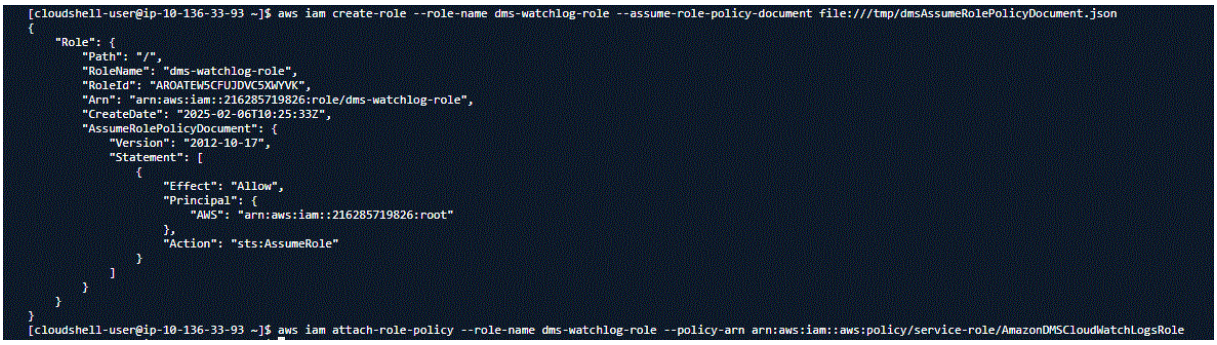
Depuis AW CLI vous devez créer le script Json qui attribue le(s) role à votre compte AWS :

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::216285719826:root"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

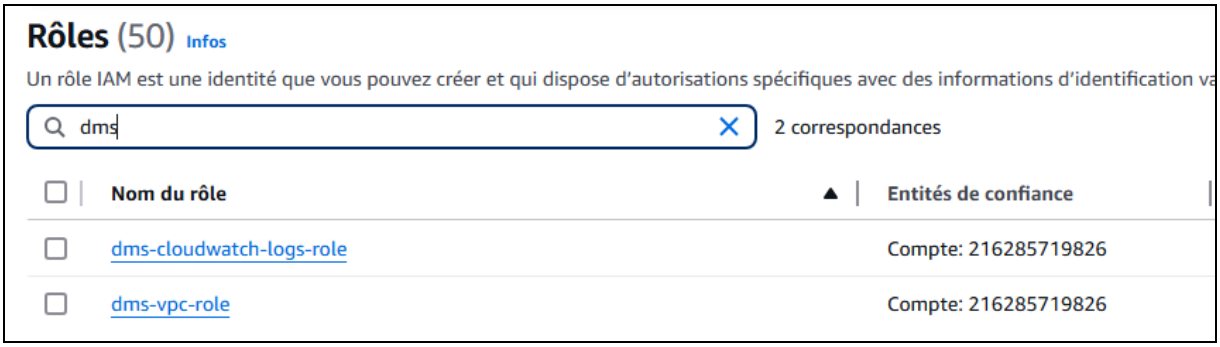
Ajout de ce nouveau rôle « dms-vpc-role » et attachement de ce dernier au compte AWS



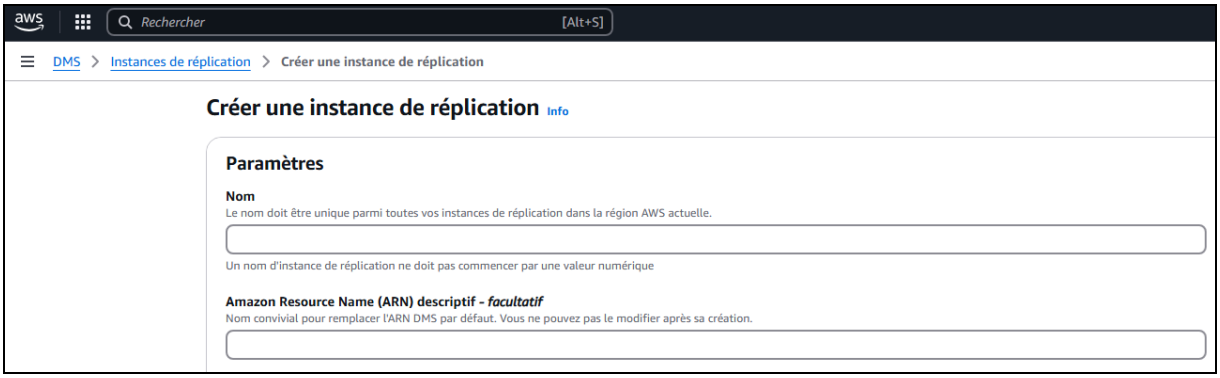
Ajout d'un nouveau rôle « dms-watchlog-role » et attachement de ce dernier au compte AWS



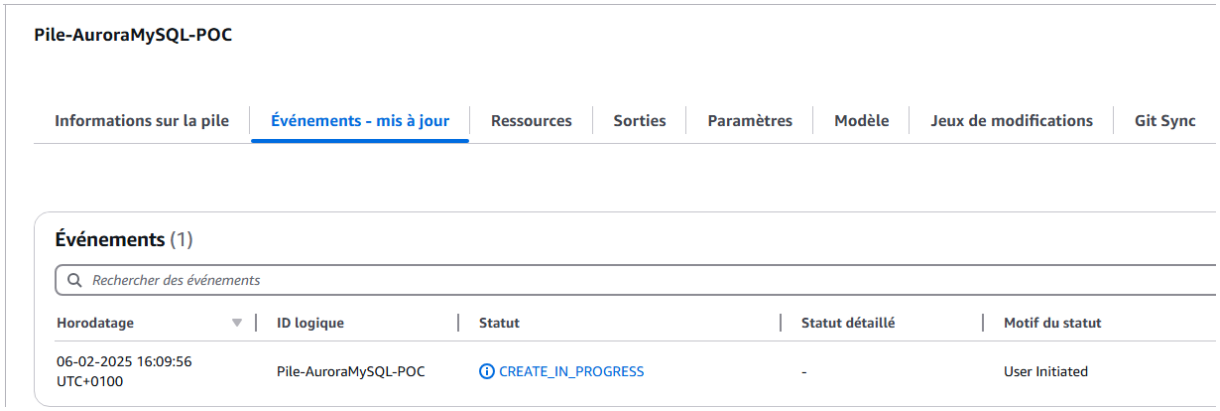
Une fois ces règles (Json) appliquées l'entité de confiance devient votre compte AWS

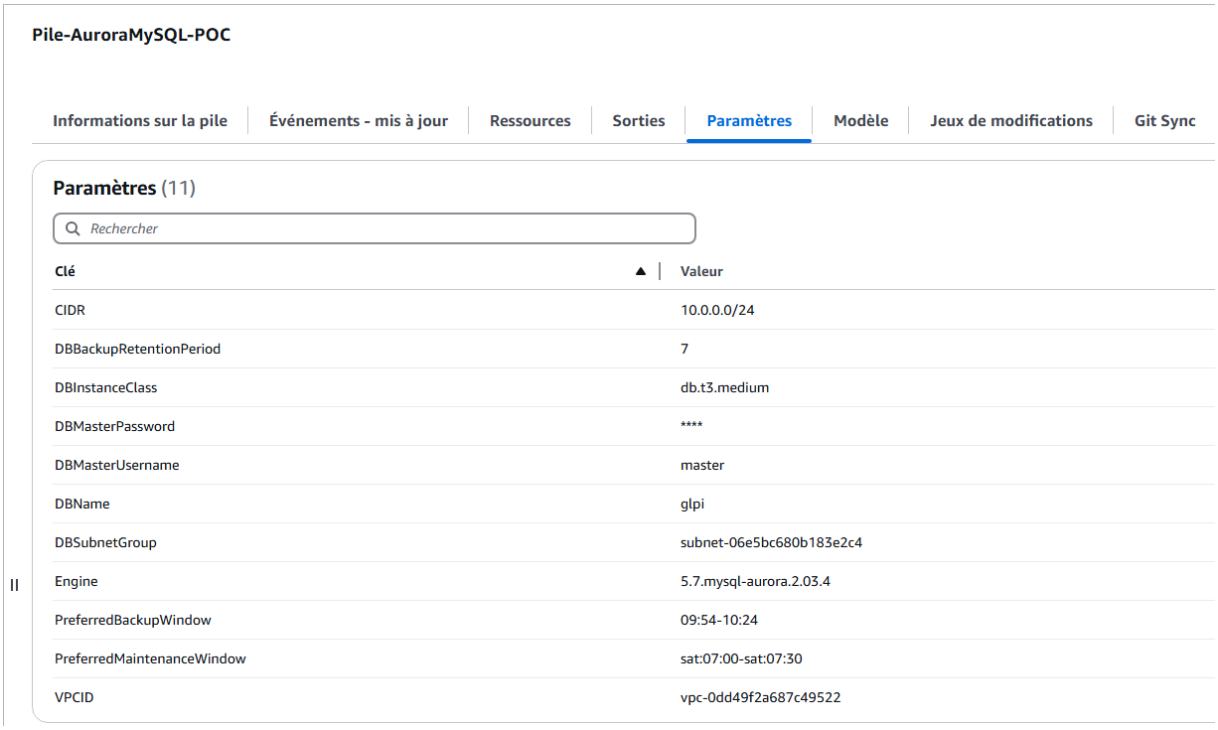


C'est seulement une fois que l'on aura ces 2 role IAM avec le compte AWS comme entité de confiance qu'il sera possible de créer une instance de réplication avec l'outil DMS sans la région concernée :



Pile construite pour DMS




Pile-AuroraMySQL-POC
Informations sur la pile
Événements - mis à jour
Ressources
Sorties
Paramètres
Modèle
Jeux de modifications
Git Sync
Paramètres (11)
Rechercher
Clé
Valeur
CIDR
10.0.0.0/24
DBBackupRetentionPeriod
7
DBInstanceClass
db.t3.medium
DBMasterPassword
****
DBMasterUsername
master
DBName
glpi
DBSubnetGroup
subnet-06e5bc680b183e2c4
Engine
5.7.mysql-aurora.2.03.4
PreferredBackupWindow
09:54-10:24
PreferredMaintenanceWindow
sat:07:00-sat:07:30
VPCID
vpc-0dd49f2a687c49522

AWS Cloudformation Pile de construction db mysql pour RDS

Événements (14)

Rechercher des événements

Horodatage	ID logique	Statut
07-02-2025 17:55:27 UTC+0100	Pile-Mariadb-POC-B	CREATE_COMPLETE
07-02-2025 17:55:27 UTC+0100	DBInstance	CREATE_COMPLETE
07-02-2025 17:44:48 UTC+0100	DBInstance	CREATE_IN_PROGRESS
07-02-2025 17:44:46 UTC+0100	DBInstance	CREATE_IN_PROGRESS
07-02-2025 17:44:45 UTC+0100	RDSMonitoringRole	CREATE_COMPLETE
07-02-2025 17:44:36 UTC+0100	DBSecurityGroup	CREATE_COMPLETE
07-02-2025 17:44:33 UTC+0100	DBParameterGroup	CREATE_COMPLETE
07-02-2025 17:44:29 UTC+0100	DBSecurityGroup	CREATE_IN_PROGRESS
07-02-2025 17:44:28 UTC+0100	RDSMonitoringRole	CREATE_IN_PROGRESS
07-02-2025 17:44:28 UTC+0100	DBParameterGroup	CREATE_IN_PROGRESS
07-02-2025 17:44:27 UTC+0100	DBParameterGroup	CREATE_IN_PROGRESS
07-02-2025 17:44:27 UTC+0100	RDSMonitoringRole	CREATE_IN_PROGRESS
07-02-2025 17:44:27 UTC+0100	DBSecurityGroup	CREATE_IN_PROGRESS
07-02-2025 17:44:24 UTC+0100	Pile-Mariadb-POC-B	CREATE_IN_PROGRESS



Mariadb10.4.32_CF.yaml